

TestPlatform

Rapport d'Execution de Campagne

ms_gestion

Projet: ms_gestion

Version: 2.5

Environnement: ms_gestion_env

Debut: 18/06/2026 16:30 | Fin: 18/06/2026 16:32

Duree: 123.5 s

Rapport genere le 18/06/2026 16:32

ECHEC

Resume executif

Total	Succes	Echecs	Erreurs	Duree
3	0	2	1	123.5 s

Taux de reussite



Evolution vs campagne precedente: DOWN 25.0 points

Campagne precedente #20

Informations du contexte

Projet	ms_gestion
Description	test

Suites

Nom	Type	Depot Git	Branche
Suite UNIT	UNIT	N/A	main

Cas de test

Titre	Type	Priorite	Risque	Script
test case add	UNIT	3	CRITICAL	IA
test add () suite test	UNIT	1	CRITICAL	IA
project create	UNIT	1	HIGH	IA

Environnement

URL Web	N/A
URL API	N/A
Repository	https://github.com/ilyeschaabani/Automatisation_intelligente_des_tests_applicatifs
Branche	main
Base de donnees	POSTGRESQL

Resultats details

Synthese par suite

Suite	Total	Succes	Echecs	Taux
Suite UNIT	3	0	3	0%

Suite UNIT

test case add

FAILURE

Type	UNIT	Duree	12.5 s
Script	IA	Erreur	[COMPILATION_ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 2.209 s <<< FAILURE! -- in suites.unit.TestCaseServiceTest

test add () suite test

ERROR

Type	UNIT	Duree	5 ms
Script	IA	Erreur	[UNKNOWN] Campaign aborted

project create

FAILURE

Type	UNIT	Duree	7.9 s
Script	IA	Erreur	[COMPILATION_ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 1.539 s <<< FAILURE! -- in suites.unit.ProjectServiceTest

Analyse IA

test case add

Erreurs extraites	<pre>[ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 2.209 s <<< FAILURE! -- in suites.unit.TestCaseServiceTest [ERROR] suites.unit.TestCaseServiceTest.testAddTestCase -- Time elapsed: 0.081 s <<< FAILURE! [ERROR] Failures: [ERROR] TestCaseServiceTest.testAddTestCase:101 [ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0 [INFO] BUILD FAILURE [ERROR] Failed to execute goal org.apache.maven.plugins:maven-surefire- plugin:3.2.5:test (default-test) on project ms_gestion: There are test failures. [ERROR] [ERROR] Please refer to C:\tmp\ms-execution\exec- 2531026973508077626\Backend\ms_gestion\target\surefire-reports for the individual test results. [ERROR] Please refer to dump files (if any exist) [date].dump, [date]-jvmRun[N].dump and [date].dumpstream. [ERROR] -> [Help 1] [ERROR] [ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch. [ERROR] Re-run Maven using the -X switch to enable full debug logging. [ERROR] [ERROR] For more information about the errors and possible solutions, please read the following articles: [ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException</pre>
Analyse IA	**Cause** : L'erreur provient de la ligne 101 (<code>`Mockito.verify(testCaseRepository, Mockito.times(1)).save(Mockito.any())`</code>), où <code>`Mockito.any()`</code> ne correspond à aucun argument concret sauvegardé, car <code>`save()`</code> a été invoqué avec un <code>`TestCase`</code> instancié dynamiquement dans <code>`.thenAnswer()`</code>, rendant la vérification trop stricte (Mockito ne parvient pas à matcher <code>`any()`</code> avec l'objet réel passé). **Correction** : Remplacer <code>`Mockito.any()`</code> par <code>`Mockito.argThat()`</code> avec prédicat spécifique, ou capturer l'argument sauvegardé dans une variable pour vérifier son contenu, ex : <pre>```java AtomicReference<TestCase> captured = new AtomicReference<>(); Mockito.when(testCaseRepository.save(Mockito.any())) .thenAnswer(inv -> { TestCase tc = inv.getArgument(0); tc.setSuite(suite); captured.set(tc); return tc; }); // ... Mockito.verify(testCaseRepository).save(captured.get()); ```</pre> **Conseil** : Lors de la vérification avec <code>`verify`</code>, éviter <code>`Mockito.any()`</code> pour les objets mutables ou construits dynamiquement ; préférer la capture d'argument (<code>`argThat`</code>, <code>`capture`</code>, ou stockage intermédiaire) pour garantir un match fiable.

test add () suite test

Erreurs extraites	
Analyse	**Cause** : La méthode <code>`testSuiteService.add(...)`</code> est appelée sans vérifier si le

IA	<code>`TestSuiteService`</code> a été correctement initialisé – la ligne <code>`@InjectMocks private TestSuiteService testSuiteService;`</code> ne peut pas instancier la classe si le constructeur n'est pas accessible (ex. privé ou sans constructeur par défaut), ou si des dépendances non mockées sont manquantes dans la hiérarchie de création. **Correction** : Vérifie que <code>`TestSuiteService`</code> a un constructeur public ou un constructeur par défaut, et que toutes ses dépendances (ex. <code>`ProjectRepository`</code>, <code>`TestSuiteRepository`</code>, <code>`ProjectMemberRepository`</code>) sont bien mockées ou fournies via <code>`@Mock`</code> (ici, la logique métier utilise probablement d'autres classes non mockées comme <code>`SecurityUtils`</code> ou <code>`ProjectService`</code>). Ajoute un constructeur explicite à <code>`TestSuiteService`</code> s'il en manque un accessible, ou utilise <code>`@Spy`</code> + <code>`@InjectMocks`</code> si besoin d'instanciation partielle. **Conseil** : Toujours inclure un test unitaire de base (ex. <code>`test_add_simpleSuccess`</code>) qui affiche une exception explicite si le service n'est pas instanciable, afin de diagnostiquer rapidement l'indisponibilité de l'objet testé avant toute assertion.
----	---

project create

Erreurs extraites	<pre>[ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 1.539 s <<< FAILURE! -- in suites.unit.ProjectServiceTest [ERROR] suites.unit.ProjectServiceTest.test_create_happy_path -- Time elapsed: 0.034 s <<< FAILURE! [ERROR] Failures: [ERROR] ProjectServiceTest.test_create_happy_path:44 Runtime Utilisateur non authentifi [ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0 [INFO] BUILD FAILURE [ERROR] Failed to execute goal org.apache.maven.plugins:maven-surefire- plugin:3.2.5:test (default-test) on project ms_gestion: There are test failures. [ERROR] [ERROR] Please refer to C:\tmp\ms-execution\exec- 2531026973508077626\Backend\ms_gestion\target\surefire-reports for the individual test results. [ERROR] Please refer to dump files (if any exist) [date].dump, [date]-jvmRun[N].dump and [date].dumpstream. [ERROR] -> [Help 1] [ERROR] [ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch. [ERROR] Re-run Maven using the -X switch to enable full debug logging. [ERROR] [ERROR] For more information about the errors and possible solutions, please read the following articles: [ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException</pre>
Analyse IA	**Cause** : La méthode <code>`ProjectService.create()`</code> tente d'accéder à l'utilisateur authentifié (probablement via <code>`SecurityContextHolder`</code>), mais dans le contexte de test Mockito, aucune authentification n'est simulée, ce qui entraîne une <code>`IllegalStateException: User not authenticated`</code>. **Correction** : Ajouter un stub d'authentification avant l'exécution du test : <pre>```java Authentication auth = new TestingAuthenticationToken("testUser", "password", Collections.singletonList(new SimpleGrantedAuthority("USER"))); SecurityContextHolder.getContext().setAuthentication(auth); ```</pre> (inséré dans la méthode <code>`test_create_happy_path`</code>, juste après la déclaration du <code>`request`</code>). **Conseil** : Toujours simuler la sécurité Spring Security dans les tests unitaires isolés en utilisant <code>`@WithMockUser`</code> ou <code>`SecurityContext`</code> manuel pour éviter les dépendances au contexte d'exécution.

Metriques

Top 5 des tests les plus lents

Test	Suite	Duree
test case add	Suite UNIT	12.5 s
project create	Suite UNIT	7.9 s
test add () suite test	Suite UNIT	5 ms

Temps d'execution par type

Type	Duree moyenne
UNIT	6.8 s

Tests instables

Aucun test instable detecte

Recommandations

- test case add:

****Cause**** : L'erreur provient de la ligne 101 (`Mockito.verify(testCaseRepository, Mockito.times(1)).save(Mockito.any())`), où `Mockito.any()` ne correspond à aucun argument concret sauvegardé, car `save()` a été invoqué avec un `TestCase` instancié dynamiquement dans `.thenAnswer()`, rendant la vérification trop stricte (Mockito ne parvient pas à matcher `any()` avec l'objet réel passé).

****Correction**** : Remplacer `Mockito.any()` par `Mockito.argThat()` avec prédicat spécifique, ou capturer l'argument sauvegardé dans une variable pour vérifier son contenu, ex :

```
```java
AtomicReference<TestCase> captured = new AtomicReference<>();
Mockito.when(testCaseRepository.save(Mockito.any()))
 .thenAnswer(inv -> {
 TestCase tc = inv.getArgument(0);
 tc.setSuite(suite);
 captured.set(tc);
 return tc;
});
// ...
Mockito.verify(testCaseRepository).save(captured.get());
```
```

****Conseil**** : Lors de la vérification avec `verify`, éviter `Mockito.any()` pour les objets mutables ou construits dynamiquement ; préférer la capture d'argument (`argThat`, `capture`, ou stockage intermédiaire) pour garantir un match fiable.

- test add () suite test:

****Cause**** : La méthode `testSuiteService.add(...)` est appelée sans vérifier si le `TestSuiteService` a été correctement initialisé — la ligne `@InjectMocks private TestSuiteService testSuiteService;` ne peut pas instancier la classe si le constructeur n'est pas accessible (ex. privé ou sans constructeur par défaut), ou si des dépendances non mockées sont manquantes dans la hiérarchie de création.

****Correction**** : Vérifie que `TestSuiteService` a un constructeur public ou un constructeur par défaut, et que toutes ses dépendances (ex. `ProjectRepository`, `TestSuiteRepository`, `ProjectMemberRepository`) sont bien mockées ou fournies via `@Mock` (ici, la logique métier utilise probablement d'autres classes non mockées comme `SecurityUtils` ou `ProjectService`). Ajoute un constructeur explicite à `TestSuiteService` s'il en manque un accessible, ou utilise `@Spy` + `@InjectMocks` si besoin d'instanciation partielle.

****Conseil**** : Toujours inclure un test unitaire de base (ex. `test_add_simpleSuccess`) qui affiche une exception explicite si le service n'est pas instanciable, afin de diagnostiquer rapidement l'indisponibilité de l'objet testé avant toute assertion.

- project create:

****Cause**** : La méthode `ProjectService.create()` tente d'accéder à l'utilisateur authentifié (probablement via `SecurityContextHolder`), mais dans le contexte de test Mockito, aucune authentification n'est simulée, ce qui entraîne une `IllegalStateException: User not authenticated`.

****Correction**** : Ajouter un stub d'authentification avant l'exécution du test :

```
```java
```

```
Authentication auth = new TestingAuthenticationToken("testUser", "password", Collections.singletonList(new
SimpleGrantedAuthority("USER")));
```

```
SecurityContextHolder.getContext().setAuthentication(auth);
```

```
```
```

(inséré dans la méthode `test\_create\_happy\_path`, juste après la déclaration du `request`).

****Conseil**** : Toujours simuler la sécurité Spring Security dans les tests unitaires isolés en utilisant `@WithMockUser` ou `SecurityContext` manuel pour éviter les dépendances au contexte d'exécution.

Annexe - Scripts generes par IA

test case add

```
package suites.unit;

import com.pfe.platform.ms_gestion.dto.request.CreateTestCaseRequest;
import com.pfe.platform.ms_gestion.dto.response.TestCaseResponse;
import com.pfe.platform.ms_gestion.entity.Project;
import com.pfe.platform.ms_gestion.entity.ProjectMember;
import com.pfe.platform.ms_gestion.entity.TestCase;
import com.pfe.platform.ms_gestion.entity.TestSuite;
import com.pfe.platform.ms_gestion.repository.ProjectMemberRepository;
import com.pfe.platform.ms_gestion.repository.TestCaseRepository;
import com.pfe.platform.ms_gestion.repository.TestSuiteRepository;
import com.pfe.platform.ms_gestion.security.SecurityUtils;
import com.pfe.platform.ms_gestion.service.LlmService;
import com.pfe.platform.ms_gestion.service.TestCaseService;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.MockedStatic;
import org.mockito.Mockito;
import org.mockito.MockitoAnnotations;
import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;
import java.util.Optional;

public class TestCaseServiceTest {

    @Mock private TestCaseRepository testCaseRepository;
    @Mock private TestSuiteRepository testSuiteRepository;
    @Mock private ProjectMemberRepository projectMemberRepository;
    @Mock private LlmService llmService;
    @InjectMocks private TestCaseService testCaseService;

    private AutoCloseable mocks;
    private MockedStatic<SecurityUtils> mockedSecurity;

    @BeforeMethod
    public void setUp() {
        mocks = MockitoAnnotations.openMocks(this);
        mockedSecurity = Mockito.mockStatic(SecurityUtils.class);
        mockedSecurity.when(SecurityUtils::getCurrentUserId).thenReturn(1L);
    }

    @AfterMethod
    public void tearDown() throws Exception {
        mockedSecurity.close();
        mocks.close();
    }

    @Test
    public void testAddTestCase() {
        Long suiteId = 1L;

        Project project = new Project();
        project.setId(10L);
        project.setName("Test Project");

        ProjectMember member = new ProjectMember();
        member.setRole(ProjectMember.Role.ADMIN);
        member.setUserId(1L);
        member.setProject(project);

        TestSuite suite = new TestSuite();
```

```

suite.setId(suiteId);
suite.setType(TestSuite.TestType.UNIT);
suite.setProject(project);

CreateTestCaseRequest request = new CreateTestCaseRequest();
request.setTitle("Login test");
request.setType("UNIT");
request.setRiskLevel("LOW");
request.setPriority(1);
request.setDescriptionAI("Test the add method");

Mockito.when(testSuiteRepository.findById(suiteId))
    .thenReturn(Optional.of(suite));
Mockito.when(projectMemberRepository.findByIdAndUserId(
    Mockito.any(), Mockito.eq(1L)))
    .thenReturn(Optional.of(member));

Mockito.when(llmService.generateTestCode(
    Mockito.any(), Mockito.any(), Mockito.any()))
    .thenReturn("public class FakeTest {}");

Mockito.when(testCaseRepository.save(Mockito.any()))
    .thenAnswer(inv -> {
        TestCase tc = inv.getArgument(0);
        tc.setSuite(suite);
        return tc;
    });

TestCaseResponse response = testCaseService.add(suiteId, request);

Assert.assertNotNull(response);
Assert.assertEquals(response.getTitle(), "Login test");
Assert.assertEquals(response.getRiskLevel(), "LOW");
Mockito.verify(testCaseRepository, Mockito.times(1)).save(Mockito.any());
// verify
Mockito.verify(llmService, Mockito.times(1))
    .generateTestCode(Mockito.any(), Mockito.any(), Mockito.any());
}
}

```

test add () suite test

```

package suites.unit;

import com.pfe.platform.ms_gestion.dto.request.CreateTestSuiteRequest;
import com.pfe.platform.ms_gestion.dto.response.TestSuiteResponse;
import com.pfe.platform.ms_gestion.entity.Project;
import com.pfe.platform.ms_gestion.entity.ProjectMember;
import com.pfe.platform.ms_gestion.entity.TestSuite;
import com.pfe.platform.ms_gestion.repository.ProjectMemberRepository;
import com.pfe.platform.ms_gestion.repository.ProjectRepository;
import com.pfe.platform.ms_gestion.repository.TestSuiteRepository;
import com.pfe.platform.ms_gestion.security.SecurityUtils;
import com.pfe.platform.ms_gestion.service.TestSuiteService;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.MockedStatic;
import org.mockito.Mockito;
import org.mockito.MockitoAnnotations;
import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;

import java.util.Optional;

public class TestSuiteServiceUnitTest {

```

```
@Mock private TestSuiteRepository testSuiteRepository;
@Mock private ProjectRepository projectRepository;
@Mock private ProjectMemberRepository projectMemberRepository;

@InjectMocks
private TestSuiteService testSuiteService; // pas de new()

private AutoCloseable mocks;
private MockedStatic<SecurityUtils> mockedSecurity;

@BeforeMethod
public void setUp() {
    mocks = MockitoAnnotations.openMocks(this);
    mockedSecurity = Mockito.mockStatic(SecurityUtils.class);
    mockedSecurity.when(SecurityUtils::getCurrentUserId).thenReturn(1L);
}

@AfterMethod
public void tearDown() throws Exception {
    mockedSecurity.close();
    mocks.close();
}

@Test
public void test_add_happyPath() {
    // Arrange
    Long projectId = 1L;

    Project project = new Project();
    project.setId(projectId);

    ProjectMember member = new ProjectMember();
    member.setUserId(1L);
    member.setRole(ProjectMember.Role.ADMIN);
    member.setProject(project);

    TestSuite saved = new TestSuite();
    saved.setId(10L);
    saved.setProject(project);

    // Mock repositories
    Mockito.when(projectRepository.findById(projectId))
        .thenReturn(Optional.of(project));
    Mockito.when(projectMemberRepository.findByProjectIdAndUserId(
        Mockito.eq(projectId), Mockito.eq(1L)))
        .thenReturn(Optional.of(member));
    Mockito.when(testSuiteRepository.save(Mockito.any()))
        .thenReturn(saved);

    // Request avec les test data générés
    CreateTestSuiteRequest request = new CreateTestSuiteRequest();
    request.setName("TestProject");
    request.setType("UNIT");
    request.setGitRepoUrl("https://github.com/testproject.git");
    request.setGitBranch("master");
    request.setModulePath("/src/main/java/com/example/test");

    // Act
    TestSuiteResponse response = testSuiteService.add(projectId, request);

    // Assert
    Assert.assertNotNull(response);
    Assert.assertEquals(response.getName(), "TestProject");
    Mockito.verify(testSuiteRepository, Mockito.times(1)).save(Mockito.any());
    Mockito.verify(projectRepository, Mockito.times(1)).findById(projectId);
}
```

project create

```
package suites.unit; //

import com.pfe.platform.ms_gestion.service.ProjectService;
import com.pfe.platform.ms_gestion.repository.ProjectRepository;
import com.pfe.platform.ms_gestion.repository.ProjectMemberRepository;
import com.pfe.platform.ms_gestion.entity.Project;
import com.pfe.platform.ms_gestion.dto.request.CreateProjectRequest;
import com.pfe.platform.ms_gestion.dto.response.ProjectResponse;
import org.mockito.*;
import org.testng.annotations.*;
import static org.testng.Assert.*;
import static org.mockito.Mockito.*;

public class ProjectServiceTest {

    private AutoCloseable mocks; //

    @Mock private ProjectRepository projectRepository;
    @Mock private ProjectMemberRepository projectMemberRepository;
    @InjectMocks private ProjectService sut;

    @BeforeMethod
    public void setUp() throws Exception {
        mocks = MockitoAnnotations.openMocks(this); //
    }

    @AfterMethod
    public void tearDown() throws Exception {
        mocks.close(); //
    }

    @Test
    public void test_create_happy_path() {
        // given
        CreateProjectRequest request = new CreateProjectRequest();
        request.setName("test");

        Project saved = new Project();
        saved.setId(1L);
        saved.setName("test"); // entité réelle, pas null
        when(projectRepository.save(any())).thenReturn(saved);

        // when
        ProjectResponse result = sut.create(request);

        // then
        assertNotNull(result);
        assertEquals(result.getName(), "test");
        verify(projectRepository, times(1)).save(any());
    }
}
```